

Server-Factory-Core-Framework

Общий движок, лежащий в основе каждого Server Factory.

Краткое описание

Core Framework — это фреймворк Kotlin, на котором базируется семейство инструментов развёртывания Server Factory. Он предоставляет общий движок и абстракции, на которых строятся такие проекты, как Mail Server Factory, позволяя каждой «фабрике» использовать единую проверенную основу вместо повторной реализации базовых примитивов развёртывания.

Краткая характеристика

Общий фреймворк Kotlin, составляющий основу экосистемы Server Factory. Он обеспечивает единый движок развёртывания, абстракции подключений и механизм выполнения шагов установки, которые используются нижестоящими фабриками (Mail Server Factory, Web Service Factory, SonarQube Factory и другими).

Подробное описание

Core Framework — это незаметный, но ключевой инженерный компонент, благодаря которому существует всё семейство Server-Factory: универсальный движок, на котором строятся все продукты-«фабрики» (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory). Подход Server Factory основан на декларативности: пользователь описывает желаемую инфраструктуру в виде конфигурации, а фабрика интерпретирует это описание, устанавливая и инициализируя ПО на целевой системе. Именно в Core Framework сосредоточена вся общая логика этого подхода: абстракции подключений и транспорта, способные работать с

любыми целевыми системами, модель шагов установки, определяющая *как* развёртывается ПО, и общая инфраструктура, которую в противном случае каждой фабрике пришлось бы реализовывать самостоятельно. Это ответ на фундаментальный вопрос, с которым рано или поздно сталкивается любая многопродуктовая инструментальная цепочка: где должен находиться общий движок? Правильный ответ на этот вопрос один раз позволяет сохранить целостность семейства, не допуская его распада на четыре схожих, но всё же разных инструмента развёртывания. Централизация этой логики в едином фреймворке Kotlin позволяет избежать дублирования кода развёртывания в разных продуктах и обеспечивает согласованность поведения: улучшение типа подключения или примитива установки в Core Framework автоматически приносит пользу всем нижестоящим фабрикам. Он почти полностью написан на Kotlin (около 990 КБ кода на Kotlin с тонким слоем на Shell), что отражает его роль библиотеки, а не набора скриптов. Репозитории нижестоящих проектов ссылаются на него как на каноническую зависимость (Parallels-Utils, Qemu-Utils, Utils и пакеты Definitions указывают на репозиторий Core Framework как на центр экосистемы). Его README намеренно минималистичен — это инфраструктура для других проектов, версия которой отслеживается через файлы `version.txt/` `version_code.txt`. Core Framework появился раньше более поздних наработок AI и является частью зрелого наследия DevOps-инструментария компании.

Зачем он был создан

Каждому инструменту развёртывания нужна одна и та же основа: способы подключения к целевым системам и шаги установки/настройки ПО. Реализация этой логики для каждого продукта отдельно привела бы к фрагментации поведения и размножению ошибок. Core Framework централизует её, обеспечивая все фабрики единым надёжным движком.

Почему это меняет правила игры

Это самая высокоэффективная точка во всём семействе: улучшение типа подключения или примитива установки здесь мгновенно распространяет корректность и новые возможности на все фабрики, позволяя всей инструментальной цепочке развиваться за счёт одного вложения. Это принцип «разработай один раз, используй везде», применённый там, где он приносит наибольшую отдачу, — на уровне фундамента автоматизации инфраструктуры, где исправление в нужном месте исправляет всё, что идёт следом.

Что нового

- Единая многоразовая инфраструктура подготовки, абстрагирующая логику подключения и установки.
- Чёткое разделение между движком (базовый фреймворк) и фабриками, специфичными для продуктов.
- Версионированное распространение (`version.txt/`
`version_code.txt`) для воспроизводимого использования.

Проблемы и решения

- **Избежание дублирования логики подготовки:** решено путём выделения общей инфраструктуры в единый фреймворк, используемый всеми фабриками.
- **Единообразие поведения в разных продуктах:** достигнуто за счёт общих абстракций, благодаря которым типы подключений и этапы установки ведут себя одинаково во всех случаях.
- **(НЕПОДТВЕРЖДЁНО):** отдельные внутренние API не документированы в публичном README; детали интерфейсов следует считать неподтверждёнными за рамками «общий фреймворк, используемый фабриками».

Технический стек (зачем и как)

- **Kotlin** — весь фреймворк (~990 КБ); язык семейства Server Factory.
- **Shell** — минимальные вспомогательные скрипты.
- **Gradle** — инструментарий сборки (соответствует подходу семейства к использованию `./gradlew`).

Примечание: GitHub обозначает репозиторий как форк в рамках организации Server-Factory. Не ориентирован на AI; представлен как основа инструментария подготовки.